# 2. Combinational Logic Design

**Text Book:**

Digital Design and Computer Architecture, 2nd Edition,
David M.H and Sarah L.H, Morgan Kaufmann, Elsevier, 2013
(ISBN: 978-0-12-394424-5)

# Introduction - Circuit

- In a digital system, a *circuit* is a network that processes *discrete-valued* variables.
- A circuit can be viewed as a **black box** (shown in **Fig 2.1**) with
  - One or more discrete-valued **input terminals**.
  - One ore more discrete-valued **output terminals**.
  - A ***functional specification*** describing the relationship between inputs and outputs.
  - A ***timing specification*** describing the delay between inputs changing and outputs responding.
- Peering inside the **black box**, the circuits are composed of ***nodes*** and ***elements*** (shown in **Figure 2.2**).
  - An **element** itself a circuit with input and output specification, e.g.: E1,.
  - **Nodes** are classified as i*nput* , *output* or *internal*, e.g.: *A*, *B*, *C*,…*Z.*

# Introduction - Circuit

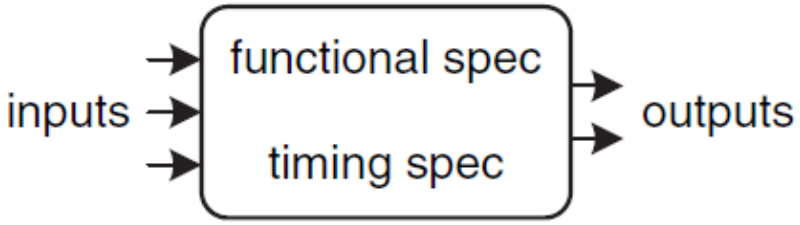



**Figure 2.1** **Circuit as a black box with inputs, outputs, and specifications**

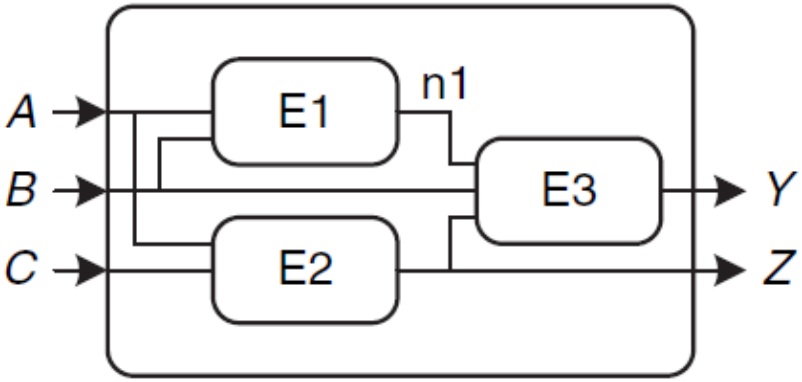



**Figure 2.2** **Elements and nodes**

# Introduction - Circuit

- Digital circuits are classified as ***combinational*** or ***sequential***.
- A **combinational circuit's** outputs depend only on the current values of the inputs;
  - in other words, *it combines the current input values to compute the output.*
  - For example, a logic gate is a combinational circuit.
- A **sequential circuit's** outputs depend on both current and previous values of the inputs;
  - in other words, it 'remembers' the past input sequence.
- A combinational circuit is ***memoryless*** (no memory), but a sequential circuit has memory.

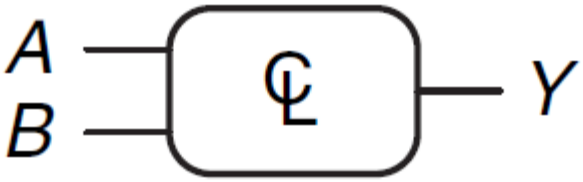
# Combinational Circuit

- The **functional specification** of a combinational circuit expresses the output values in terms of the current input values.
- The **timing specification** of a combinational circuit consists of lower and upper bounds on the delay from input to output.

# Combinational Circuit

- **Figure 2.3** shows a combinational circuit with *two inputs* and *one output*. On the left of the figure are the inputs, ***A*** and ***B***, and on the right is the output, ***Y***.
- The symbol $\varphi$ inside the box indicates that it is implemented using only *combinational logic*.
- In this example, the function ***F*** is specified to be **OR**: ***Y*** = ***F***(***A***, ***B***) = ***A + B***. In words, we say "the output ***Y*** is a function of the two inputs, ***A*** and ***B***", namely ***Y*** = ***A OR B***.

# Combinational Circuit




$$Y = F(A, B) = A + B$$

**Figure 2.3 Combinational logic circuit**

# Combinational Circuit

- **Figure 2.4** shows two possible implementations for the combinational logic circuit in **Figure 2.3**.

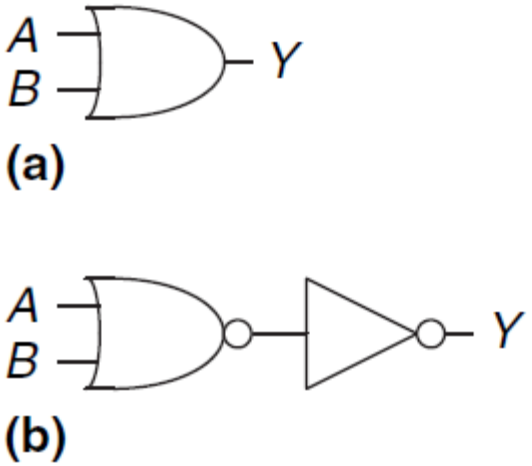



**Figure 2.4 Two OR implementations**

# Combinational Circuit

- ***The rules of combinational composition***
  - A circuit is combinational if it consists of interconnected circuit elements such that
    - ▶ Every circuit element is itself combinational.
    - ▶ Every node of the circuit is either designated as an input to the circuit or connects to exactly *one output terminal of a circuit element.*
    - ▶ The circuit contains *no cyclic paths*: every path through the circuit visits each circuit node at most once.

# Combinational Circuit

- **Example 2.1** Which of the circuits in **Figure 2.7** are combinational circuits according to the rules of combinational composition?

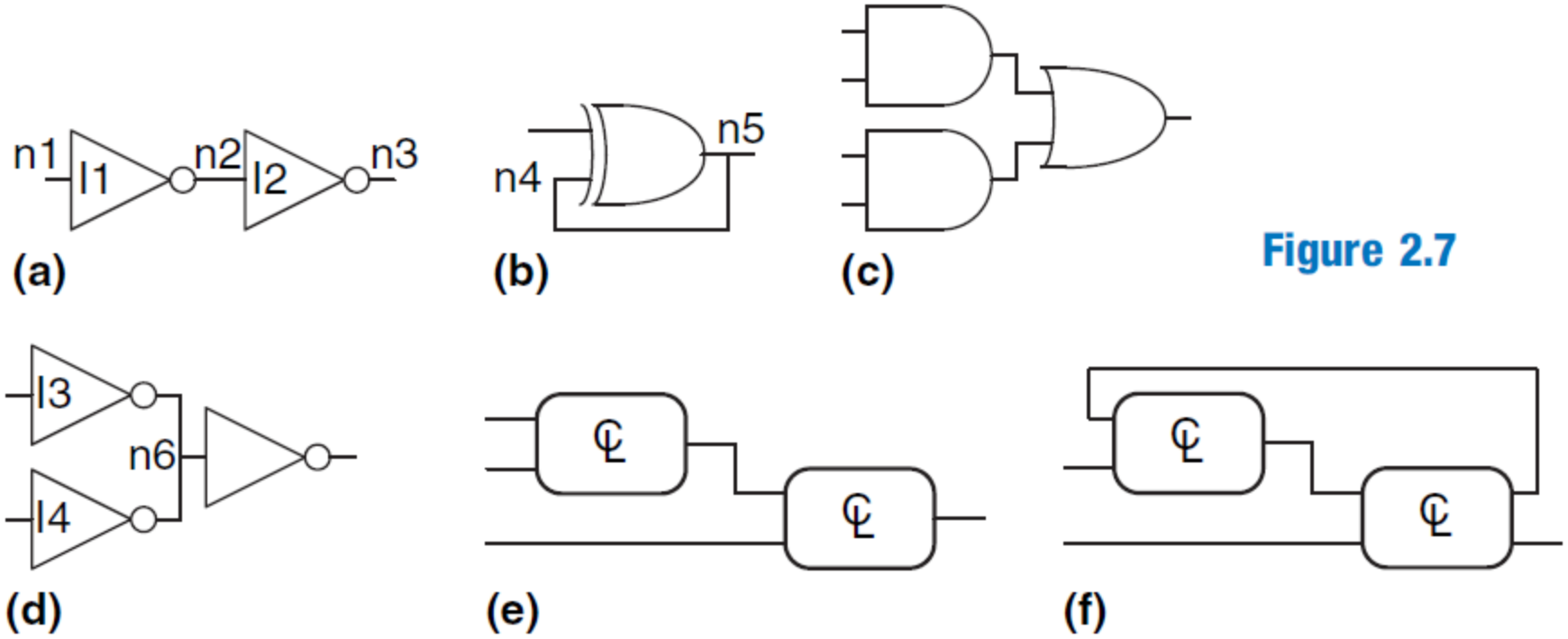



Figure 2.7

$\overline{A}$

# Boolean Equations

- The functional specification of a combinational circuit is usually expressed as a ***truth table*** or ***a Boolean equation***.
- **Boolean equations** deal with variables that are either TRUE or FALSE, so they are perfect for describing digital logic.

**Terminology**

- The ***complement*** of a variable *A* is its *inverse* $\overline{A}$
- The variable or its complement is called a ***literal***. For example, *A*, $\overline{A}$, *B*, and $\overline{B}$ are literals.

**Product or implicant**

- The **AND** of one or more literals is called a ***product*** or an ***implicant*,** for example, $\overline{A}BC$ and $A\overline{B}C$ are *all implicants for a function of three variables.*

# Boolean Equations

**Minterm**

- A *minterm* is a **product** involving all of the inputs to the function, for example, $A\overline{B}\overline{C}$ is a minterm for a function of the three variables *A*, *B*, and *C*, but $\overline{AB}$ is not, because it does not involve *C*.

**Sum**

- The **OR** of one or more literals is called a ***sum***, for example *A*+*B*+*C* is the *sum* of three literals *A*, *B* and *C*.

**Maxterm**

- A *maxterm* is a **sum** involving all of the inputs to the function. *A* + $\overline{B}$ + *C* is a *maxterm* for a function of the three variables *A*, *B*, and *C*.

# Boolean Equations

- The ***order of operations*** is important when interpreting Boolean equations. Does $Y = A + BC$ mean $Y = (A$ OR $B)$ AND $C$ or $Y = A$ OR$(B$ AND $C)$?
- In Boolean equations, **NOT** has the highest precedence, followed by **AND**, then **OR**. Just as in ordinary equations, products are performed before sums.
- Therefore, the equation is read as $Y=A$ OR $(B$ AND $C)$. Equation 2.1 gives another example of order of operations.

$$\overline{A}B + BC\overline{D} = ((\overline{A})B) + (BC(\overline{D})) \qquad (2.1)$$

# Sum-of-products form

- A truth table of ***N*** inputs contains $2^N$ rows, one for each possible value of the inputs. Each row in a truth table is associated with a *minterm* that is TRUE for that row. **Figure 2.8** shows a truth table of two inputs, A and B. Each row shows its corresponding *minterm* (where $m_1$ is a ***true*** minterm).

| *A* | *B* | *Y* | minterm | minterm name |
|---|---|---|---|---|
| 0 | 0 | 0 | $\overline{A}\ \overline{B}$ | $m_0$ |
| 0 | 1 | 1 | $\overline{A}\ B$ | $m_1$ |
| 1 | 0 | 0 | $A\ \overline{B}$ | $m_2$ |
| 1 | 1 | 0 | $A\ B$ | $m_3$ |

**Figure 2.8 Truth table and minterms**

# Sum-of-products form

- We can write a *Boolean equation* for any **truth table** by summing each of the *minterms* for which the output, *Y*, is TRUE. For example, in **Figure 2.8**, there is only one row (or minterm) for which the output *Y* is TRUE, shown circled in blue. Thus, *Y* = $\overline{AB}$
- **Figure 2.9** shows a **truth table** with more than one row in which the output is TRUE. Taking the sum of each of the circled *minterms* (only TRUE) gives:

$$Y = \overline{A}B + AB$$

- This is called the ***sum-of-products*** canonical form of a function because it is the ***sum*** (OR) of ***products*** (ANDs forming ***minterms***).

# Sum-of-products form

| A | B | Y | minterm | minterm name |
|---|---|---|---|---|
| 0 | 0 | 0 | $\overline{A}\ \overline{B}$ | $m_0$ |
| 0 | 1 | 1 | $\overline{A}\ B$ | $m_1$ |
| 1 | 0 | 0 | $A\ \overline{B}$ | $m_2$ |
| 1 | 1 | 1 | $A\ B$ | $m_3$ |

**Figure 2.9 Truth table with multiple TRUE minterms**

# Sum-of-products form

- The ***sum-of-products*** canonical form can also be written in sigma notation using the summation symbol, **Σ**. With this notation, the function from **Figure 2.9** would be written as:

$$F(A,B) = \Sigma(m_1, m_3) \qquad (2.2)$$

  - The ***sum-of-products*** form provides a Boolean equation for any truth table with any number of variables. **Figure 2.12** shows a random three input truth table. The ***sum-of-products*** form of the logic function is

$$Y = \overline{A}\,\overline{B}\,\overline{C} + A\,\overline{B}\,\overline{C} + A\,\overline{B}C \quad \text{or} \qquad (2.3)$$

$$Y = \Sigma(0, 4, 5)$$

# Sum-of-products form

| A | B | C | Y |
|---|---|---|---|
| 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | 0 |

**Figure 2.12 Random three-input truth table**

# Product-of-Sums Form

- An alternative way of expressing Boolean functions is the **product-of-sums** canonical form.
- Each row of a *truth table* corresponds to a ***maxterm*** that is **FALSE** for that row. For example, the ***maxterm*** for the first row of a two-input *truth table* is (*A* + *B*) in **Figure 2.13** because (*A* + *B*) is FALSE when *A* = 0, *B* = 0.
- We can write a *Boolean equation* for any circuit directly from the truth table as the AND of each of the *maxterms* for which the output is **FALSE**.
- The **product-of-sums** canonical form can also be written in **pi** notation using the product symbol, **Π.**

# Product-of-Sums Form

**Example 2.3** PRODUCT-OF-SUMS FORM

Write an equation in product-of-sums form for the truth table in **Figure 2.13**.

**Solution:**

The truth table has two rows in which the output is FALSE. Hence, the function can be written in product-of-sums form as

$$Y = (A + B)(\overline{A} + B)$$

or, using pi notation,

$Y = \Pi(M_0, M_2)$ or $Y = \Pi(0, 2)$.

# Product-of-Sums Form

| A | B | Y | maxterm | maxterm name |
|---|---|---|---|---|
| 0 | 0 | 0 | $A + B$ | $M_0$ |
| 0 | 1 | 1 | $A + \overline{B}$ | $M_1$ |
| 1 | 0 | 0 | $\overline{A} + B$ | $M_2$ |
| 1 | 1 | 1 | $\overline{A} + \overline{B}$ | $M_3$ |

**Figure 2.13 Truth table with multiple FALSE maxterms**

# Comparison

- **sum-of-products** produces a shorter equation when the output is **TRUE** on only a few rows of a truth table.
- **product-of-sums** is simpler when the output is **FALSE** on only a few rows of a truth table.

# Boolean Algebra

- **Boolean algebra** is used to simplify Boolean equations.
- The rules of Boolean algebra are much like those of ordinary algebra but are in some cases simpler, because variables have only two possible values: **0** or **1**.
- Boolean algebra is based on a set of **axioms** that we assume are correct.
- **Table 2.1** states the **axioms** of Boolean algebra.
- From these axioms, *we prove all the theorems of Boolean algebra, because they teach us how to simplify logic to produce smaller and less costly circuits*.
- Axioms and theorems of Boolean algebra obey the principle of *duality*.

# Boolean Algebra-Axioms

**Table 2.1 Axioms of Boolean algebra**

| | Axiom | | Dual | Name |
|---|---|---|---|---|
| A1 | $B = 0$ if $B \neq 1$ | A1′ | $B = 1$ if $B \neq 0$ | Binary field |
| A2 | $\overline{0} = 1$ | A2′ | $\overline{1} = 0$ | NOT |
| A3 | $0 \bullet 0 = 0$ | A3′ | $1 + 1 = 1$ | AND/OR |
| A4 | $1 \bullet 1 = 1$ | A4′ | $0 + 0 = 0$ | AND/OR |
| A5 | $0 \bullet 1 = 1 \bullet 0 = 0$ | A5′ | $1 + 0 = 0 + 1 = 1$ | AND/OR |

These five axioms and their duals define Boolean variables and the meanings of NOT, AND, and OR.

# Boolean Algebra-Theorems of One Variable

**Theorems of one variable:** Theorems T1 to T5 in **Table 2.2** describe how to simplify equations involving one variable.

**Table 2.2 Boolean theorems of one variable**

| | Theorem | | Dual | Name |
|---|---|---|---|---|
| T1 | $B \bullet 1 = B$ | T1′ | $B + 0 = B$ | Identity |
| T2 | $B \bullet 0 = 0$ | T2′ | $B + 1 = 1$ | Null Element |
| T3 | $B \bullet B = B$ | T3′ | $B + B = B$ | Idempotency |
| T4 | | $\overline{\overline{B}} = B$ | | Involution |
| T5 | $B \bullet \overline{B} = 0$ | T5′ | $B + \overline{B} = 1$ | Complements |

# Boolean Algebra-Theorems of Several Variables

**Theorems of Several Variables:**

Theorems T6 to T12 in **Table 2.3** describe how to simplify equations involving more than one Boolean variable.

# Boolean Algebra-Theorems of Several Variables

**Table 2.3 Boolean theorems of several variables**

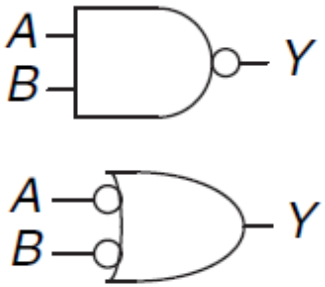
| | Theorem | | Dual | Name |
|---|---|---|---|---|
| T6 | $B \bullet C = C \bullet B$ | T6′ | $B + C = C + B$ | Commutativity |
| T7 | $(B \bullet C) \bullet D = B \bullet (C \bullet D)$ | T7′ | $(B + C) + D = B + (C + D)$ | Associativity |
| T8 | $(B \bullet C) + (B \bullet D) = B \bullet (C + D)$ | T8′ | $(B + C) \bullet (B + D) = B + (C \bullet D)$ | Distributivity |
| T9 | $B \bullet (B + C) = B$ | T9′ | $B + (B \bullet C) = B$ | Covering |
| T10 | $(B \bullet C) + (B \bullet \overline{C}) = B$ | T10′ | $(B + C) \bullet (B + \overline{C}) = B$ | Combining |
| T11 | $(B \bullet C) + (\overline{B} \bullet D) + (C \bullet D) = B \bullet C + \overline{B} \bullet D$ | T11′ | $(B + C) \bullet (\overline{B} + D) \bullet (C + D) = (B + C) \bullet (\overline{B} + D)$ | Consensus |
| T12 | $\overline{B_0 \bullet B_1 \bullet B_2 \ldots} = (\overline{B}_0 + \overline{B}_1 + \overline{B}_2 \ldots)$ | T12′ | $\overline{B_0 + B_1 + B_2 \ldots} = (\overline{B}_0 \bullet \overline{B}_1 \bullet \overline{B}_2 \ldots)$ | De Morgan's Theorem |

# De Morgan's theorem

- According to **De Morgan's theorem**, a **NAND** gate is equivalent to an **OR** gate with **inverted inputs**.
- Similarly, a **NOR** gate is equivalent to an **AND** gate with **inverted inputs**.
- **Figure 2.19** shows these De Morgan equivalent gates for NAND and NOR gates.
- The two symbols shown for each function are called **duals**.
  - They are logically equivalent and can be use interchangeably.

# De Morgan's theorem

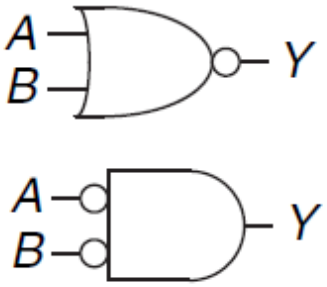
**NAND**




$Y = \overline{AB} = \overline{A} + \overline{B}$

| A | B | Y |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

**NOR**




$Y = \overline{A + B} = \overline{A}\,\overline{B}$

| A | B | Y |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 0 |

**Figure 2.19 De Morgan equivalent gates**

# De Morgan's theorem

**Example 2.4** DERIVE THE PRODUCT-OF-SUMS FORM

**Figure 2.20** shows the truth table for a Boolean function *Y* and its complement $\overline{Y}$: Using De Morgan's Theorem, *derive the product-of-sums canonical form of Y from the sum-of-products form of* $\overline{Y}$ *:*

**Solution:**

**Figure 2.21** shows the minterms (circled) contained in *Y*: The sum-of-products canonical form of *Y* is

$$\overline{Y} = \overline{A}\,\overline{B} + \overline{A}B \qquad (2.4)$$

Taking the complement of both sides and applying De Morgan's Theorem twice, we get:

$$\overline{\overline{Y}} = Y = \overline{\overline{A}\,\overline{B} + \overline{A}B} = (\overline{\overline{A}\,\overline{B}})(\overline{\overline{A}B}) = (A + B)(A + \overline{B}) \qquad (2.5)$$

# De Morgan's theorem

| $A$ | $B$ | $Y$ | $\overline{Y}$ |
|---|---|---|---|
| 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 1 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 0 |

**Figure 2.20** Truth table showing $Y$ and $\overline{Y}$

| $A$ | $B$ | $Y$ | $\overline{Y}$ | minterm |
|---|---|---|---|---|
| 0 | 0 | 0 | 1 | $\overline{A}\,\overline{B}$ |
| 0 | 1 | 0 | 1 | $\overline{A}\,B$ |
| 1 | 0 | 1 | 0 | $A\,\overline{B}$ |
| 1 | 1 | 1 | 0 | $A\,B$ |

**Figure 2.21** Truth table showing minterms for $\overline{Y}$

# Proving a Theorem

- Prove the consensus theorem, T11, from Table 2.3.

$$BC + \overline{B}D + CD = BC + \overline{B}D$$

| $B$ | $C$ | $D$ | $BC + \overline{B}D + CD$ | $BC + \overline{B}D$ |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 | 0 |
| 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 |

The theorem is proved!

# Simplifying Equation

- The **theorems of Boolean algebra** help us simplify Boolean equations. For example, consider the sum-of-products expression from the truth table of **Figure 2.9**:

$$\overline{A}\,\overline{B} + A\overline{B}$$

- By **Theorem T10**, the equation simplifies to $Y = \overline{B}$ : This may have been obvious looking at the truth table.

**Minimize Equation**: $\overline{A}\overline{B}\overline{C} + A\overline{B}\overline{C} + A\overline{B}C$

**Solution**: Using the *idempotency theorem*, we can duplicate terms as many times as we want: $B = B + B + B + B$…. Using this principle, we simplify the equation completely to its two prime *implicants*, $\overline{B}\overline{C} + A\overline{B}$ ,as shown in **Table 2.5** and **Figure 2.25** shows schematic the equation.

# Simplifying Equation

**Table 2.4 Equation minimization**

| Step | Equation | Justification |
|---|---|---|
| | $\overline{A}\,B\,\overline{C} + A\overline{B}\,\overline{C} + A\overline{B}C$ | |
| 1 | $\overline{B}\,\overline{C}(\overline{A} + A) + A\overline{B}C$ | T8: Distributivity |
| 2 | $\overline{B}\,\overline{C}(1) + A\overline{B}C$ | T5: Complements |
| 3 | $\overline{B}\,\overline{C} + A\overline{B}C$ | T1: Identity |

**Table 2.5 Improved equation minimization**

| Step | Equation | Justification |
|---|---|---|
| | $\overline{A}\,B\,\overline{C} + A\overline{B}\,\overline{C} + A\overline{B}C$ | |
| 1 | $\overline{A}\,B\,\overline{C} + A\overline{B}\,\overline{C} + A\overline{B}\,\overline{C} + A\overline{B}C$ | T3: Idempotency |
| 2 | $\overline{B}\,\overline{C}(\overline{A} + A) + A\overline{B}(\overline{C} + C)$ | T8: Distributivity |
| 3 | $\overline{B}\,\overline{C}(1) + A\overline{B}(1)$ | T5: Complements |
| 4 | $\overline{B}\,\overline{C} + A\overline{B}$ | T1: Identity |

# Simplifying Equation

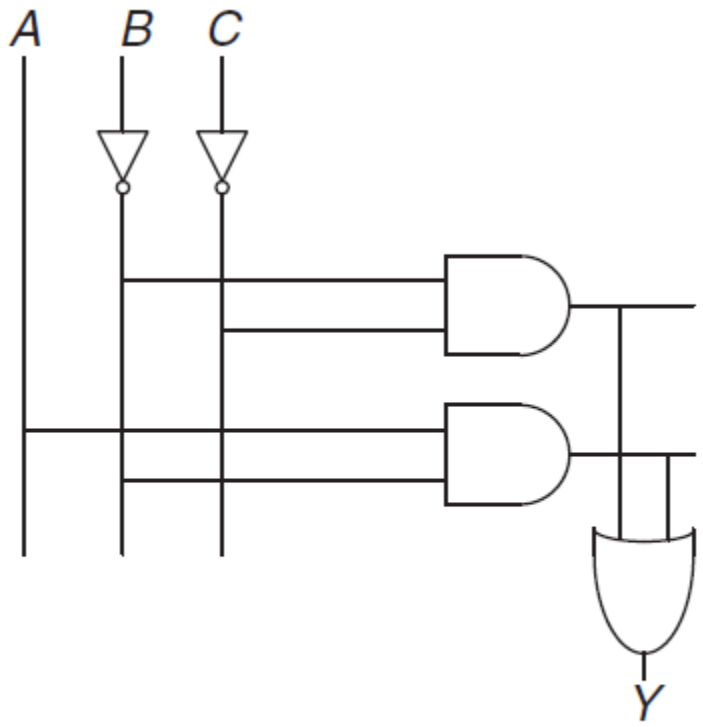
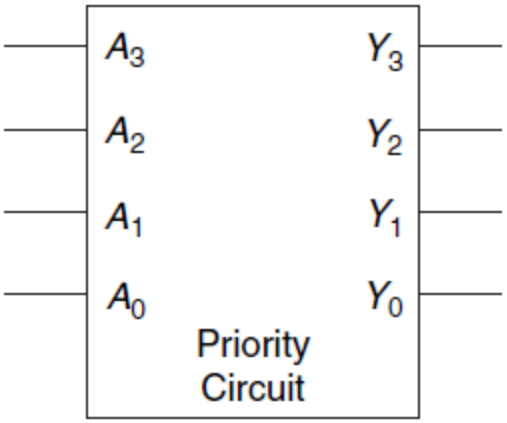



**Figure 2.25** Schematic of $Y = \overline{B}\,\overline{C} + A\overline{B}$

# Multiple Output Circuits

- The dean, the department chair, the teaching assistant, and the dorm social chair each use the auditorium from time to time. Unfortunately, they occasionally conflict, leading to disasters such as the one that occurred when the dean's fundraising meeting with crusty trustees happened at the same time as the dorm's BTB1 party. Alyssa P. Hacker has been called in to design a room reservation system.
- The system has four inputs, A3, . . . , A0, and four outputs, Y3, . . . , Y0. These signals can also be written as A3:0 and Y3:0. Each user asserts her input when she requests the auditorium for the next day. The system asserts at most one output, granting the auditorium to the highest priority user. The dean, who is paying for the system, demands highest priority (3). The department chair(2), teaching assistant(1), and dorm social chair(0) have decreasing priority. **Write a truth table and Boolean equations for the system. Sketch a circuit that performs this function.**




| $A_3$ | $A_2$ | $A_1$ | $A_0$ | $Y_3$ | $Y_2$ | $Y_1$ | $Y_0$ |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 |
| 0 | 1 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 | 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 0 | 1 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 | 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 | 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 0 | 0 | 1 | 0 | 0 | 0 |
| 1 | 1 | 0 | 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 1 | 0 | 1 | 0 | 0 | 0 |
| 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 |

**Figure 2.27** **Priority circuit**

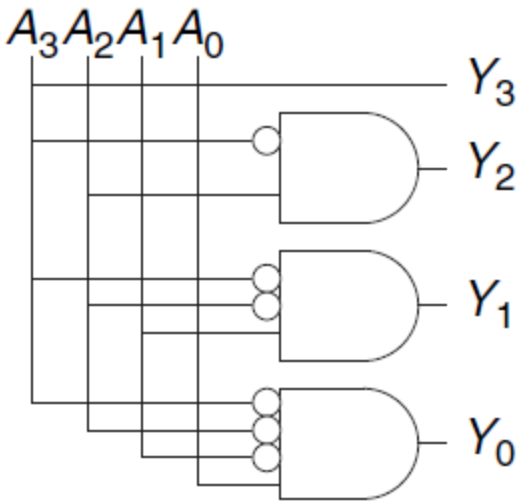



**Figure 2.28** Priority circuit schematic

| $A_3$ | $A_2$ | $A_1$ | $A_0$ | $Y_3$ | $Y_2$ | $Y_1$ | $Y_0$ |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | X | 0 | 0 | 1 | 0 |
| 0 | 1 | X | X | 0 | 1 | 0 | 0 |
| 1 | X | X | X | 1 | 0 | 0 | 0 |

**Figure 2.29** Priority circuit truth table with don't cares (X's)

# Multi-level Combinational Logic

- Logic in **sum-of-products** form is called **two-level logic** because it consists of literals connected to a level of AND gates connected to a level of OR gates.
- **Figure 2.30** shows the truth table for a **three-input XOR** with the rows circled that produce **TRUE outputs**.
- From the **truth table**, we read off a Boolean equation in sum-of-products form in **Equation 2.6**.

$$Y = \overline{A}\,\overline{B}C + \overline{A}B\overline{C} + A\overline{B}\,\overline{C} + ABC \quad (2.6)$$

- *Unfortunately, there is no way to simplify this equation into fewer implicants*.
- **Prove the equation (2.6) by perfect induction method and show** $Y = A \oplus B \oplus C$

# Multi-level Combinational Logic

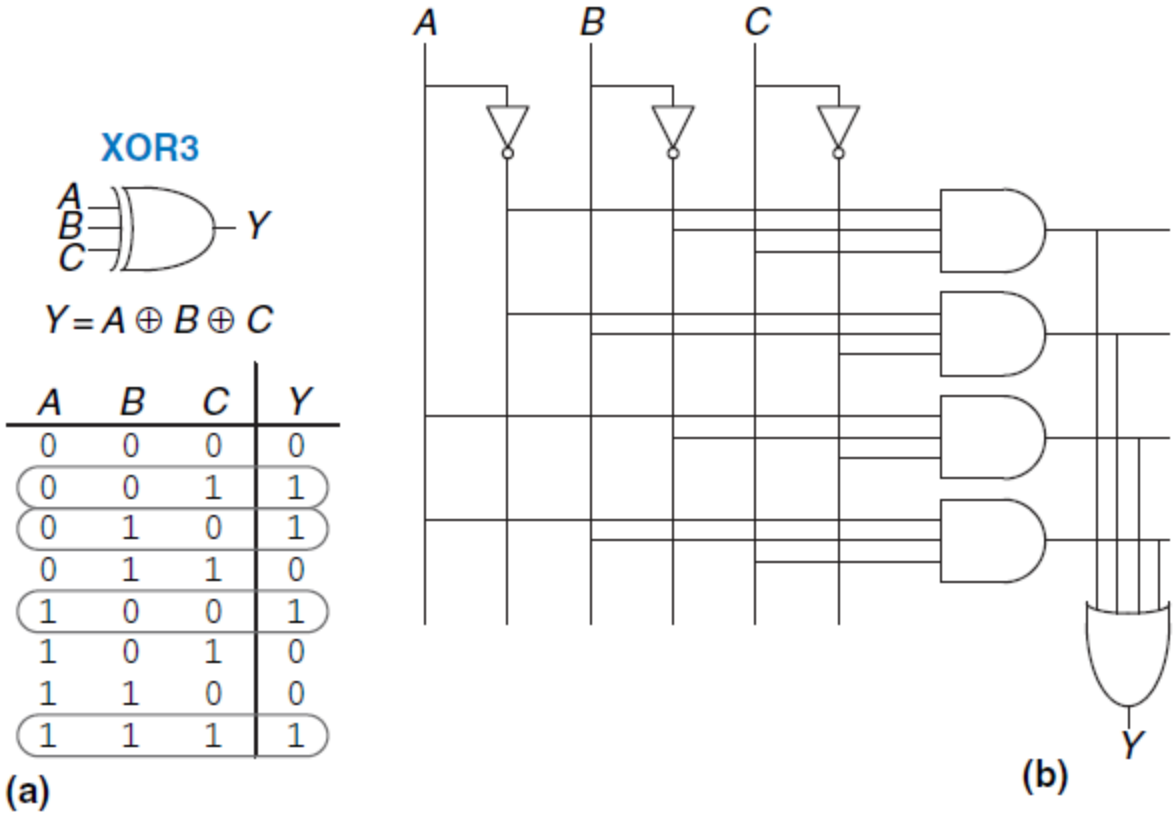



**Figure 2.30** Three-input XOR:
(a) functional specification and (b) two-level logic implementation

# Bubble Pushing

- CMOS circuits design prefer NANDs and NORs over ANDs and ORs.
- **Figure 2.33** shows a multilevel circuit whose function is not immediately clear by inspection.
- **Bubble pushing** is a helpful way to redraw these circuits so that the bubbles cancel out and the function can be more easily determined.

**The guidelines for bubble pushing are as follows:**

- ▶ Begin at the output of the circuit and work toward the inputs.
- ▶ Push any bubbles on the final output back toward the inputs so that you can read an equation in terms of the output (for example *Y*), instead of the complement of the output $\overline{Y}$

# Bubble Pushing

- Working backward, draw each gate in a form so that bubbles cancel. If the current gate has an input bubble, draw the preceding gate with an output bubble. If the current gate does not have an input bubble, draw the preceding gate without an output bubble.
- **Figure 2.34** shows how to redraw **Figure 2.33** according to the bubble pushing guidelines. Starting at the output *Y,* the NAND gate has a bubble on the output that we wish to eliminate.
- We push the output bubble back to form an OR with inverted inputs, shown in **Figure 2.34(a).**

# Bubble Pushing

- Working to the left, the rightmost gate has an input bubble that cancels with the output bubble of the middle NAND gate, so no change is necessary, as shown in **Figure 2.34(b)**.
- The middle gate has no input bubble, so we transform the leftmost gate to have no output bubble, as shown in **Figure 2.34(c)**.
- Now all of the bubbles in the circuit cancel except at the inputs, so the function can be read by inspection in terms of ANDs and ORs of true or complementary inputs:

$$Y = \overline{A}\,\overline{B}C + \overline{D}$$

- For emphasis of this last point, **Figure 2.35** shows a circuit logically equivalent to the one in **Figure 2.34**.

# Bubble Pushing

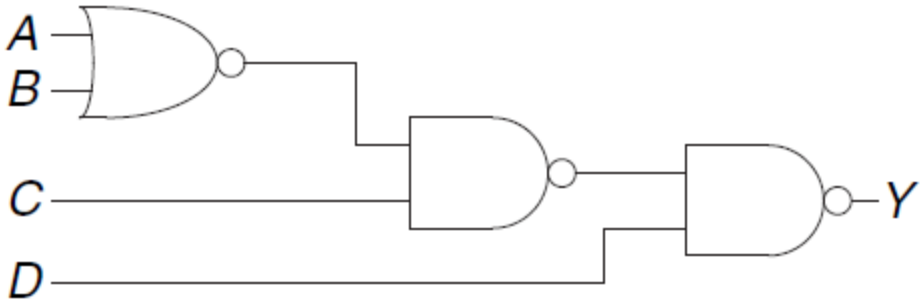



**Figure 2.33 Multilevel circuit using NANDs and NORs**

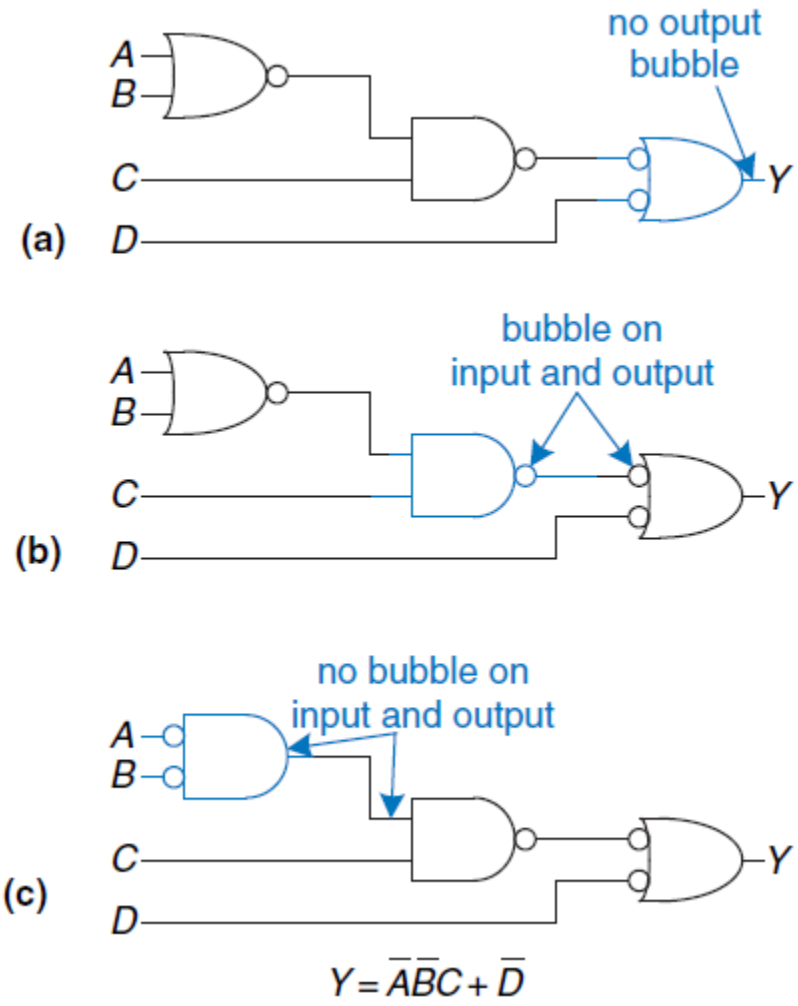



**Figure 2.34** **Bubble-pushed circuit**

# Bubble Pushing

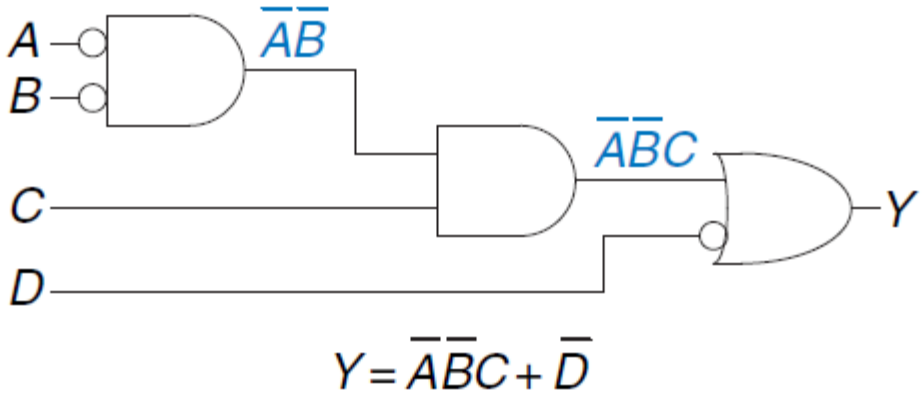



**Figure 2.35** Logically equivalent bubble-pushed circuit

# Bubble Pushing

**Example 2.8** BUBBLE PUSHING FOR CMOS LOGIC

- Most designers think in terms of AND and OR gates, but suppose you would like to implement the circuit in **Figure 2.36** in CMOS logic, which favors NAND and NOR gates.

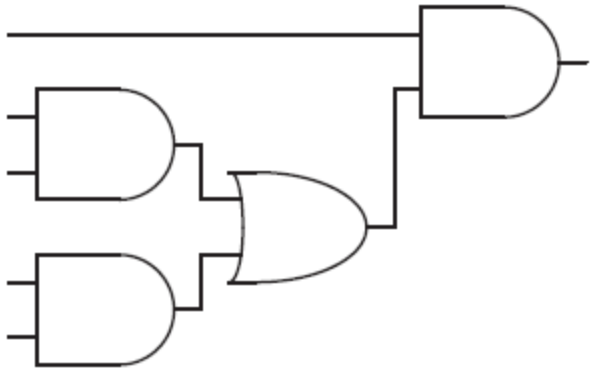


**Figure 2.36 Circuit using ANDs and ORs**

# Bubble Pushing

**Solution:**

- **A brute force solution** is to just replace each AND gate with a NAND and an inverter, and each OR gate with a NOR and an inverter, as shown in **Figure 2.37**.
  - This requires eight gates.
  - Notice that the *inverter* is drawn with the bubble on the front rather than back, to emphasize how the bubble can cancel with the preceding inverting gate.
- **For a better solution**, observe that bubbles can be added to the output of a gate and the input of the next gate without changing the function, as shown in **Figure 2.38(a).**
- The final AND is converted to a NAND and an inverter, as shown in **Figure 2.38(b).** This solution requires only five gates.

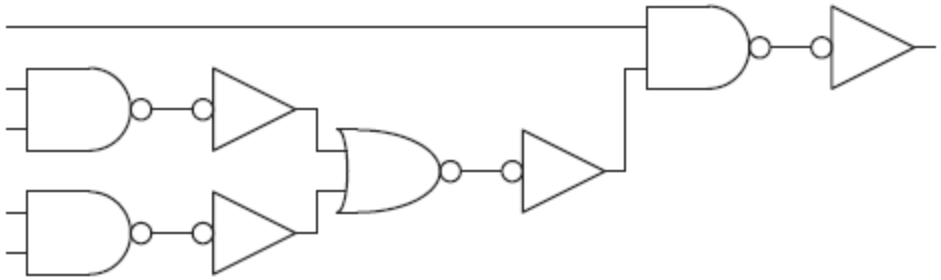


**Figure 2.37** Poor circuit using NANDs and NORs

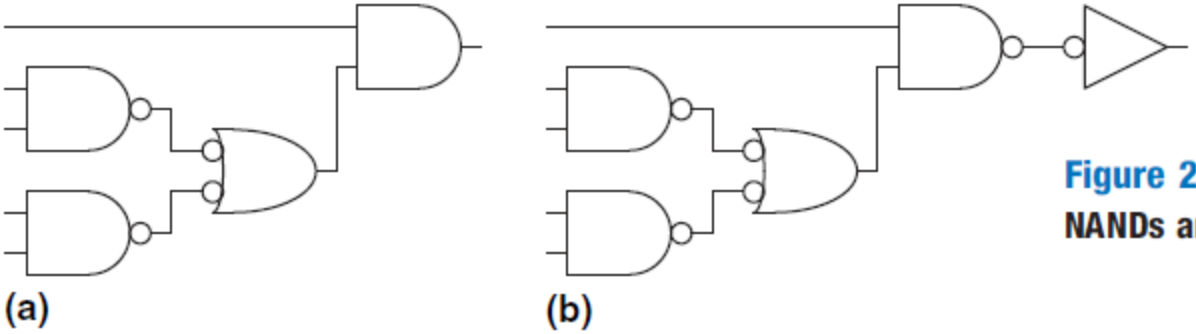



**Figure 2.38** Better circuit using NANDs and NORs

# Limitation of Boolean Algebra

- **Boolean algebra** is limited to 0's and 1's. However, real circuits can also have illegal and floating values, represented symbolically by X and Z.
  - The symbol X indicates that the circuit node has an unknown or illegal value.
  - The symbol Z indicates that a node is being driven neither HIGH nor LOW.

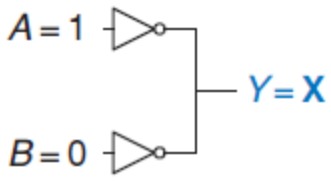
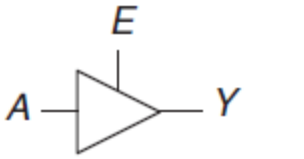
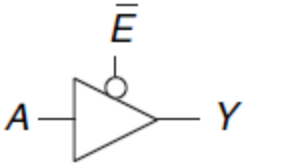



**Figure 2.39** Circuit with contention




| E | A | Y |
|---|---|---|
| 0 | 0 | Z |
| 0 | 1 | Z |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

**Figure 2.40** Tristate buffer




| $\bar{E}$ | A | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | Z |
| 1 | 1 | Z |

**Figure 2.41** Tristate buffer with active low enable

# Karnaugh Maps (K-maps)

- **Karnaugh maps** (or **K-maps**) are a graphical method for *simplifying* ***Boolean equations***.
  - They were invented in 1953 by Maurice Karnaugh, a telecommunications engineer at Bell Labs.
- **K-maps** work well for problems with up to **four variables**. More important, they give insight into manipulating **Boolean equations**
  - Recall that logic minimization involves combining terms. Two terms containing an *implicant* ***P*** and the true and complementary forms of some variable ***A*** are combined to eliminate $A: PA + P\overline{A} = P$
  - ***Karnaugh maps*** *make these combinable terms easy to see by putting them next to each other in a grid*.

# Karnaugh Maps (K-maps)

- **Figure 2.43** shows the *truth table* and *K-map* for a *three-input function*.
  - The top row of the ***K*-map** gives the four possible values for the ***A*** and ***B*** inputs.
  - The left column gives the two possible values for the ***C*** input.
  - Each square in the ***K-map*** corresponds to a row in the truth table and contains the value of the output ***Y*** for that row.
  - For example, the top left square corresponds to the first row in the truth table and indicates that the output value ***Y* =1** when ***ABC* = 000.**
  - Just like each row in a truth table, each square in a ***K-map*** represents a single *minterm*. For the purpose of explanation, **Figure 2.43(c)** shows the minterm corresponding to each square in the ***K-map.***

| A | B | C | Y |
|---|---|---|---|
| 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | 0 |

(a)

| Y / C \ AB | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | 1 | 0 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 |

(b)

| Y / C \ AB | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | $\bar{A}\bar{B}\bar{C}$ | $\bar{A}B\bar{C}$ | $AB\bar{C}$ | $A\bar{B}\bar{C}$ |
| 1 | $\bar{A}\bar{B}C$ | $\bar{A}BC$ | $ABC$ | $A\bar{B}C$ |

(c)

**Figure 2.43** Three-input function: (a) truth table, (b) K-map, (c) K-map showing minterms

| Y / C \ AB | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | 1 | 0 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 |

**Figure 2.44** K-map minimization

***K-maps*** help us do this simplification graphically by circling 1's in adjacent squares, as shown in **Figure 2.44**. For each circle, we write the corresponding implicant.

# Karnaugh Maps (K-maps)

- You may have noticed that the ***A*** and ***B*** combinations in the top row are in a peculiar order: **00, 01, 11, 10**. This order is called a **Gray code**.
  - It differs from ordinary binary order (00, 01, 10, 11) in that adjacent entries differ only in a single variable. For example, 01 : 11 only changes
- The ***K-map*** also “wraps around.” The squares on the far right are effectively adjacent to the squares on the far left, in that they differ only in one variable, ***A***.
  - In other words, you could take the map and roll it into a cylinder, then join the ends of the cylinder to form a torus (i.e., a donut),
- And still ***guarantee that adjacent squares would differ only in one variable***.

# Logic Minimization with K-Maps

- **K-maps** provide an easy visual way to minimize logic.
- Simply circle all the rectangular blocks of 1's in the map, using the fewest possible number of circles.
- Each circle should be as large as possible. Then read off the *implicants* that were circled.
- Each circle on the **K-map** represents an implicant.
- The largest possible circles are prime implicants.

# Logic Minimization with K-Maps

- For example, in the *K-map* of **Figure 2.44**, $\overline{A}\,\overline{B}\,\overline{C}$ and $\overline{A}\,\overline{B}C$ are implicants, **but not prime implicants**.
- **Only $\overline{A}\,\overline{B}$ is a prime implicant in that K-map**.
- Rules for finding a minimized equation from a *K-map* are as follows:
  - Use the fewest circles necessary to cover all the 1's.
  - All the squares in each circle must contain 1's.
  - Each circle must span a rectangular block that is a power of 2 (i.e., 1, 2, or 4) squares in each direction.
  - Each circle should be as large as possible.
  - A circle may wrap around the edges of the *K-map*.
  - A 1 in a *K-map* may be circled multiple times if doing so allows fewer circles to be used.

# Example 2.9 Minimization of a Three-Variable Function Using a K-map

- Suppose we have the function $Y = F(A, B, C)$ with the *K-map* shown in **Figure 2.45**. Minimize the equation using the K-map.

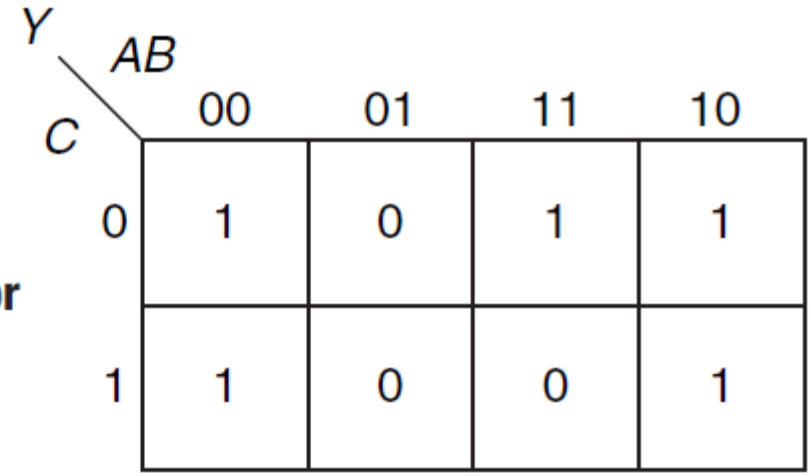



**Figure 2.45 K-map for Example 2.9**

**Solution**: Circle the 1's in the *K-map* using as few circles as possible, as shown in **Figure 2.46**. Each circle in the *K-map* represents a ***prime implicant***, and the dimension of each circle is a power of two ($2 \times 1$ and $2 \times 2$). We form the prime implicant for each circle by writing those variables that appear in the circle only in true or only in complementary form.

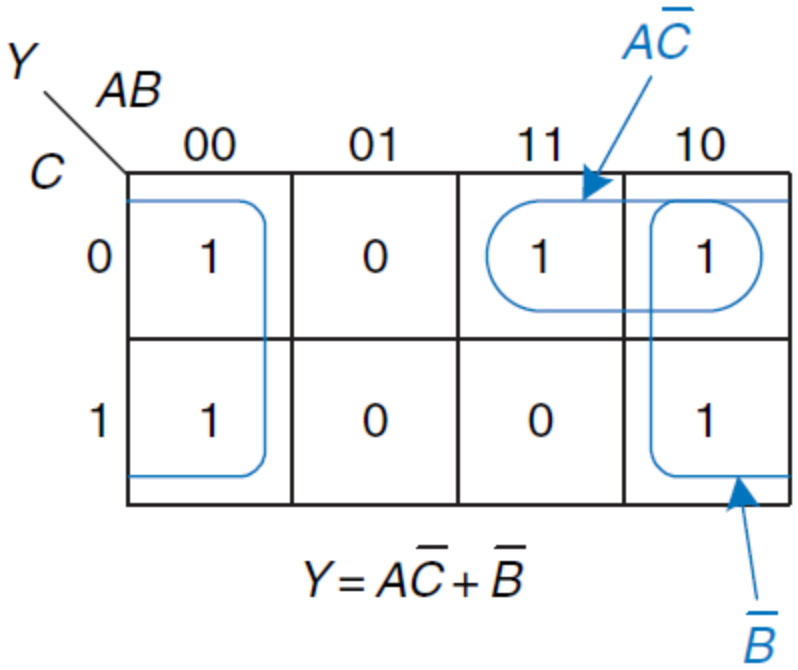



**Figure 2.46 Solution for Example 2.9**

# Example 2.10 Seven-Segment Display Decoder

- A seven-segment display decoder takes a **4-bit data** input *D*3:*D*0 and produces seven outputs to control light-emitting diodes to display a digit from 0 to 9. The seven outputs are often called segments ***a*** through ***g***, or $S_a$–$S_g$, as defined in **Figure 2.47**. The digits are shown in **Figure 2.48**. *Write a truth table for the outputs, and use K-maps to find Boolean equations for outputs $S_a$ and $S_b$*. Assume that illegal input values (10–15) produce a blank readout.

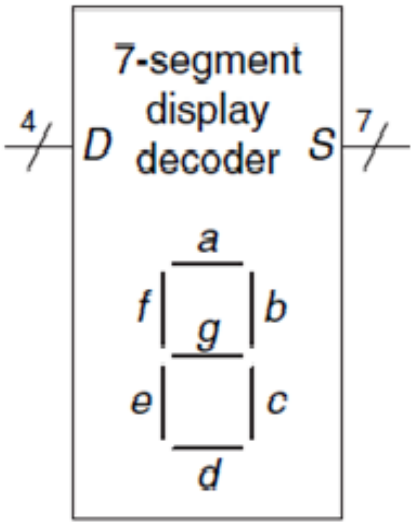



**Figure 2.47** Seven-segment display decoder icon

# Example 2.10 Seven-Segment Display Decoder

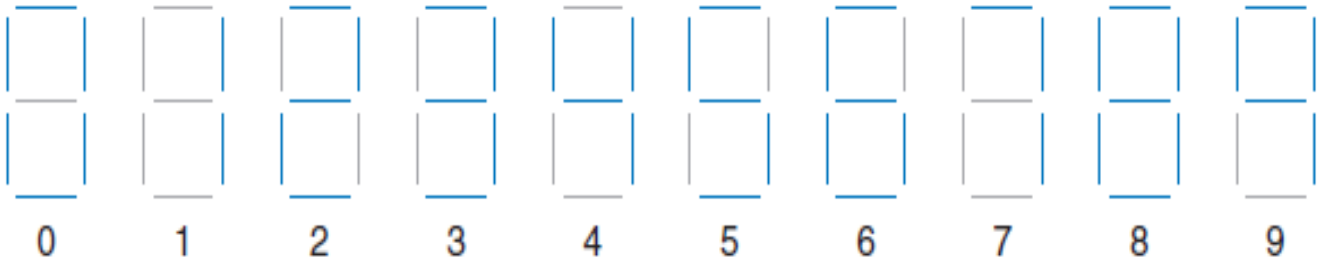



**Figure 2.48** Seven-segment display digits

**Solution**: The truth table is given in **Table 2.6**. For example, an input of 0000 should turn on all segments except $S_g$.

**Table 2.6 Seven-segment display decoder truth table**

| $D_{3:0}$ | $S_a$ | $S_b$ | $S_c$ | $S_d$ | $S_e$ | $S_f$ | $S_g$ |
|---|---|---|---|---|---|---|---|
| 0000 | 1 | 1 | 1 | 1 | 1 | 1 | 0 |
| 0001 | 0 | 1 | 1 | 0 | 0 | 0 | 0 |
| 0010 | 1 | 1 | 0 | 1 | 1 | 0 | 1 |
| 0011 | 1 | 1 | 1 | 1 | 0 | 0 | 1 |
| 0100 | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| 0101 | 1 | 0 | 1 | 1 | 0 | 1 | 1 |
| 0110 | 1 | 0 | 1 | 1 | 1 | 1 | 1 |
| 0111 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |
| 1000 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 1001 | 1 | 1 | 1 | 0 | 0 | 1 | 1 |
| others | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

- Each of the seven outputs is an independent function of four variables. The K-maps for outputs $S_a$ and $S_b$ are shown in **Figure 2.49**. Remember that adjacent squares may differ in only a single variable, so we label the rows and columns in Gray code order: 00, 01, 11, 10.

$S_a$

| $D_{1:0}$ \ $D_{3:2}$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 1 | 0 | 0 | 1 |
| 01 | 0 | 1 | 0 | 1 |
| 11 | 1 | 1 | 0 | 0 |
| 10 | 1 | 1 | 0 | 0 |

$S_b$

| $D_{1:0}$ \ $D_{3:2}$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 1 | 1 | 0 | 1 |
| 01 | 1 | 0 | 0 | 1 |
| 11 | 1 | 1 | 0 | 0 |
| 10 | 1 | 0 | 0 | 0 |

**Figure 2.49** Karnaugh maps for $S_a$ and $S_b$

- Next, circle the prime implicants. Use the fewest number of circles necessary to cover all the 1's. A circle can wrap around the edges (vertical and horizontal), and a 1 may be circled more than once. **Figure 2.50** shows the prime implicants and the simplified Boolean equations.

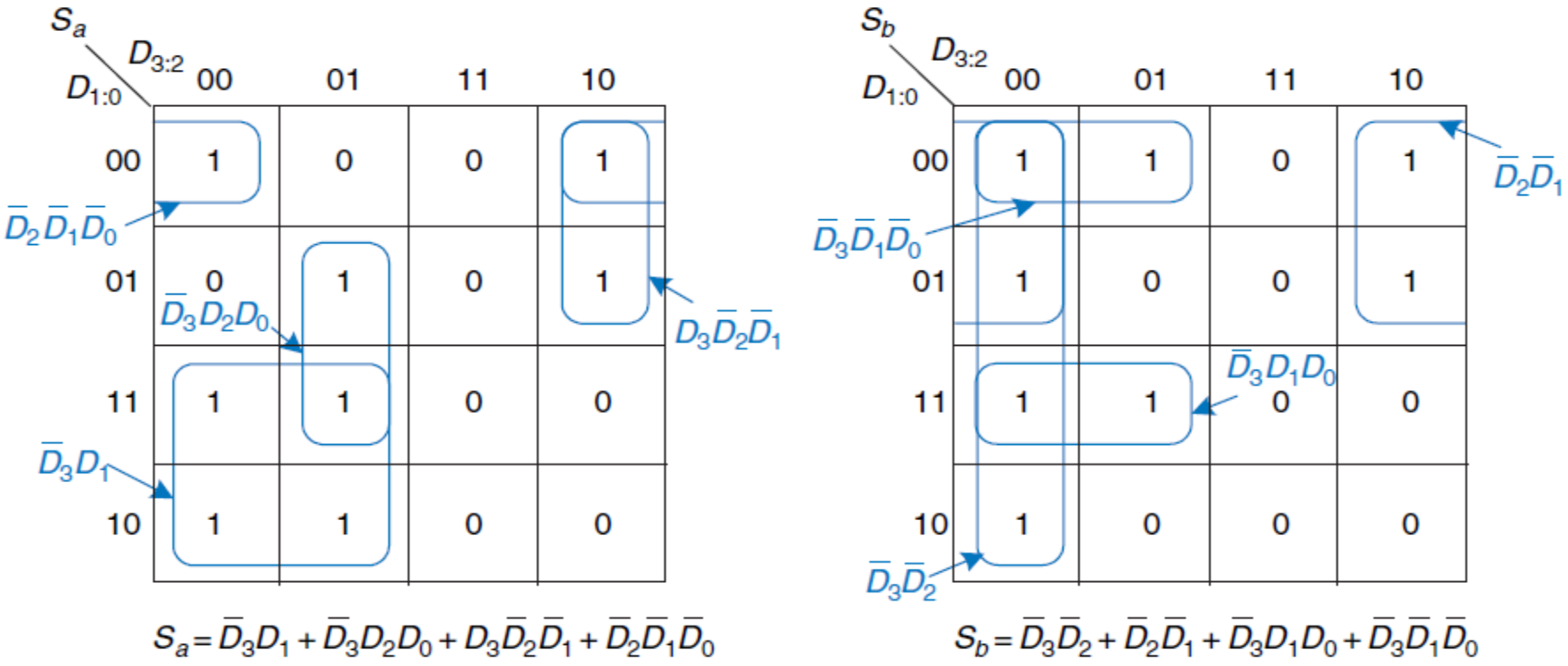
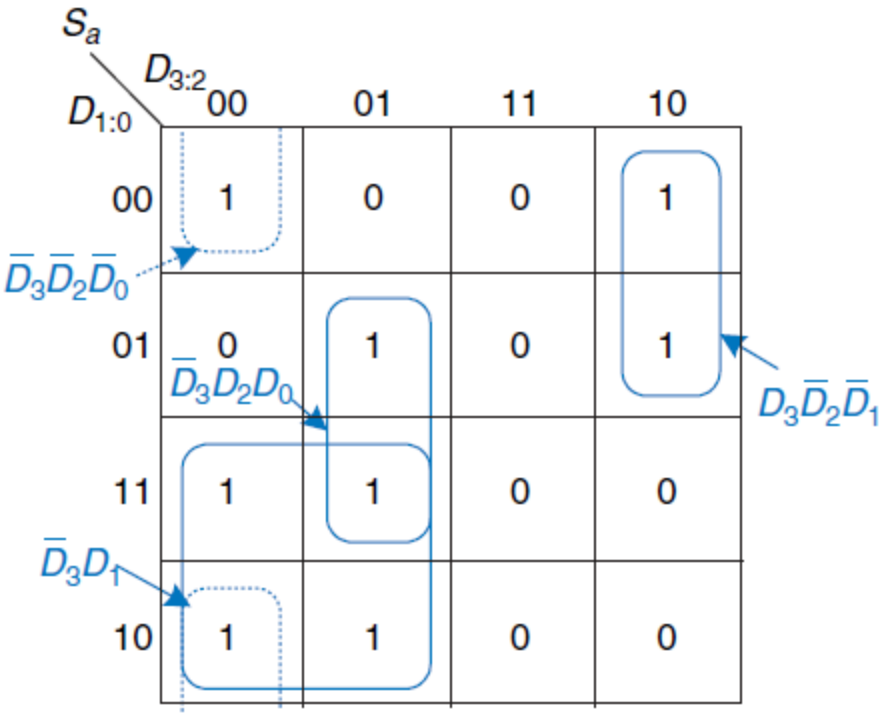


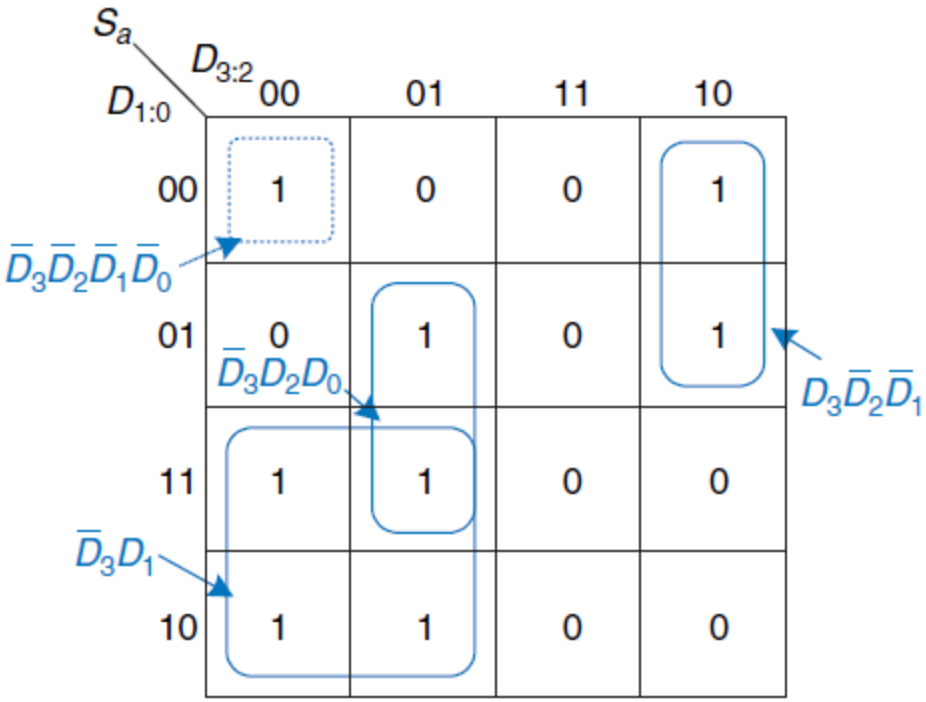

**Figure 2.50** K-map solution for Example 2.10

- Note that the minimal set of prime implicants is not unique. For example, the 0000 entry in the $S_a$ K-map was circled along with the 1000 entry to produce the $\overline{D}_2\overline{D}_1\overline{D}_0$ minterm. The circle could have included the 0010 entry instead, producing a $\overline{D}_3\overline{D}_2\overline{D}_0$ minterm, as shown with dashed lines in **Figure 2.51**.
- **Figure 2.52** illustrates a common error in which a nonprime implicant was chosen to cover the 1 in the upper left corner. This minterm, $\overline{D}_3\overline{D}_2\overline{D}_1\overline{D}_0$, gives a sum-of-products equation that is not minimal. The minterm could have been combined with either of the adjacent ones to form a larger circle, as was done in the previous two figures.




$$S_a = \bar{D}_3D_1 + \bar{D}_3D_2D_0 + D_3\bar{D}_2\bar{D}_1 + \bar{D}_3\bar{D}_2\bar{D}_0$$

**Figure 2.51** Alternative K-map for $S_a$ showing different set of prime implicants




$$S_a = \bar{D}_3D_1 + \bar{D}_3D_2D_0 + D_3\bar{D}_2\bar{D}_1 + \bar{D}_3\bar{D}_2\bar{D}_1\bar{D}_0$$

**Figure 2.52** Alternative K-map for $S_a$ showing incorrect nonprime implicant

# Don’t Cares

- **Don’t cares** also appear in truth table outputs where the output value is unimportant or the corresponding input combination can never happen.
- Such outputs can be treated as either 0’s or 1’s at the designer’s discretion.
- In a K-map, **X**’s allow for even more logic minimization. They can be circled if they help cover the 1’s with fewer or larger circles, but they do not have to be circled if they are not helpful.

**Example 2.11** SEVEN-SEGMENT DISPLAY DECODER WITH DON’T CARES

  – Repeat **Example 2.10** if we don’t care about the output values for illegal input values of 10 to 15.

# Don’t Cares

- The ***K-map*** is shown in **Figure 2.53** with **X** entries representing **don’t care**.
- Because don’t cares can be 0 or 1, we circle a don’t care if it allows us to cover the 1’s with fewer or bigger circles. Circled don’t cares are treated as 1’s, whereas un-circled don’t cares are 0’s.
- Observe how a $2 \times 2$ square wrapping around all four corners is circled for segment $S_a$.
- Use of **don’t cares** simplifies the logic substantially.

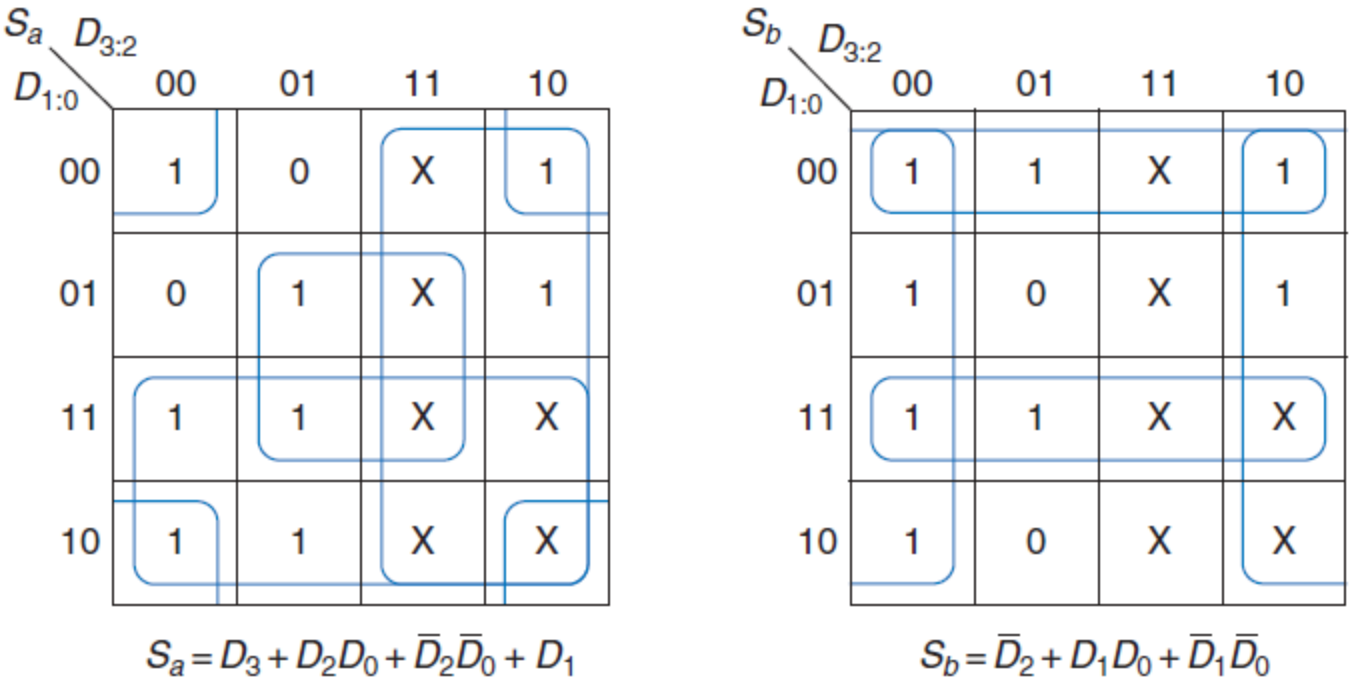
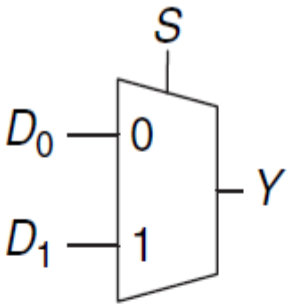



**Figure 2.53** **K-map solution with don't cares**

# Combinational Building Blocks

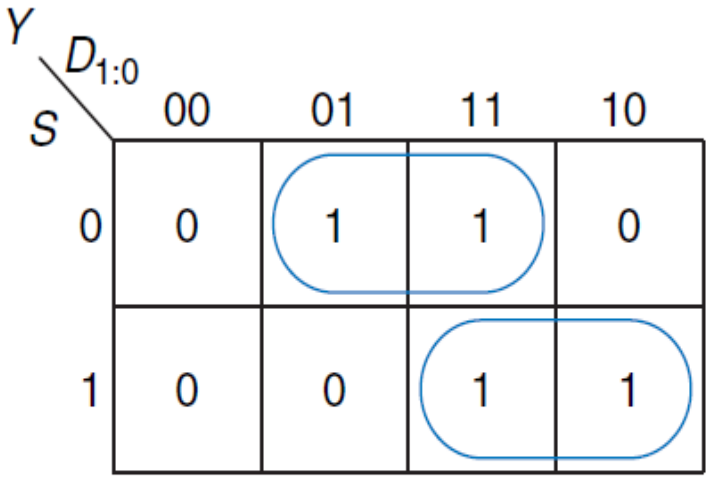
**Multiplexers**

- Multiplexers are among the most commonly used combinational circuits. They choose an output from among several possible inputs based on the value of a select signal. A multiplexer is sometimes affectionately called a **mux**.
- **Figure 2.54** shows the schematic and truth table for a 2:1 multiplexer with two data inputs D0 and D1, a select input S, and one output Y.The multiplexer chooses between the two data inputs based on the select: if S = 0, Y =D0, and if S = 1, Y =D1. S is also called a **control signal** because it controls what the multiplexer does. A 2:1 multiplexer can be built from sum-of-products logic as shown in **Figure 2.55**.




| S | $D_1$ | $D_0$ | Y |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

**Figure 2.54** 2:1 multiplexer symbol and truth table




$$Y = D_0\bar{S} + D_1S$$

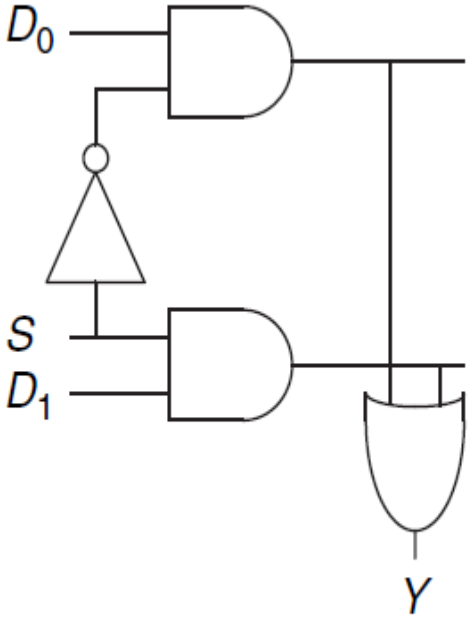



**Figure 2.55** 2:1 multiplexer implementation using two-level logic

# Combinational Building Blocks

**Wider Multiplexers**

- A 4:1 multiplexer has four data inputs and one output, as shown in **Figure 2.57**.

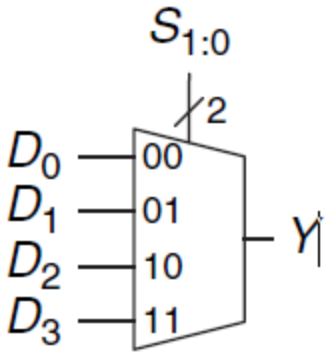



**Figure 2.57 4:1 multiplexer**

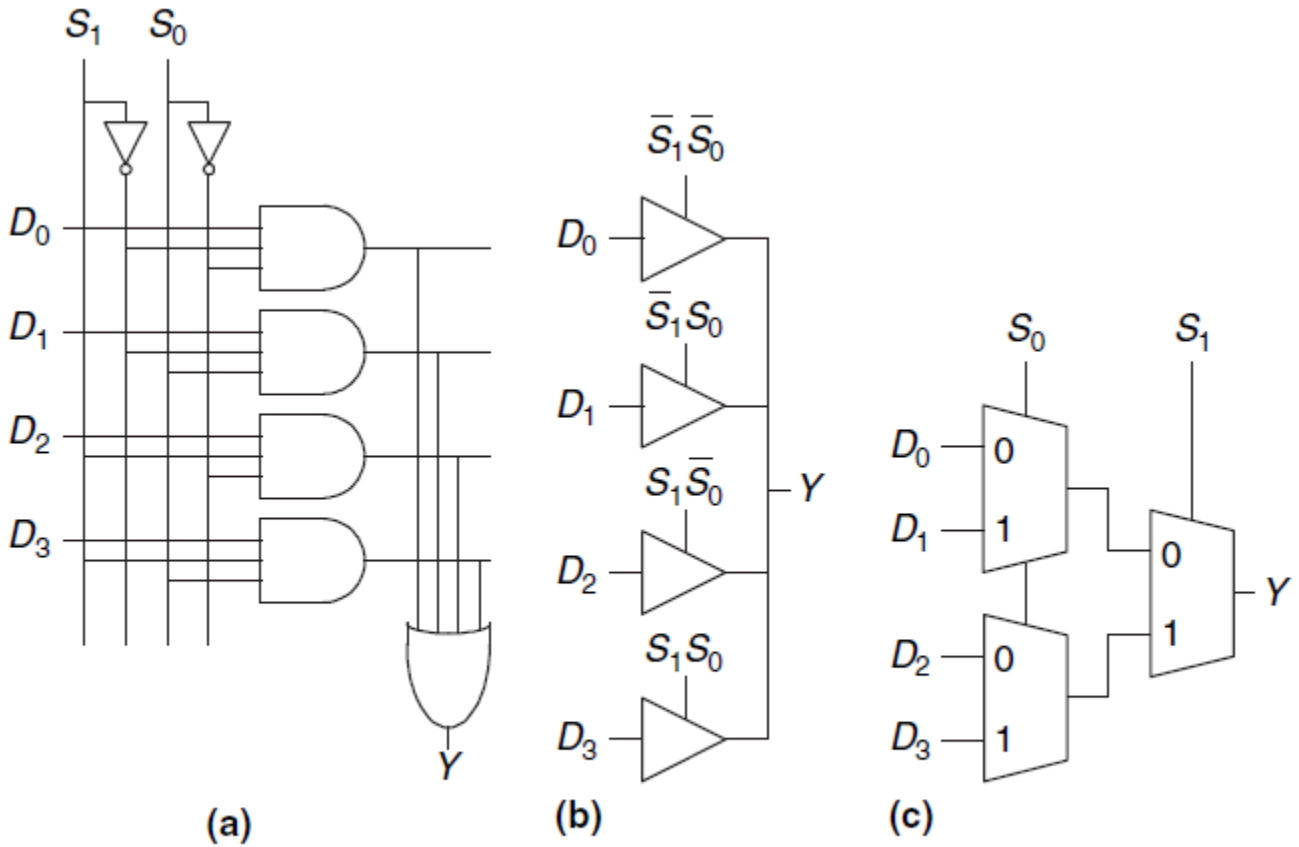
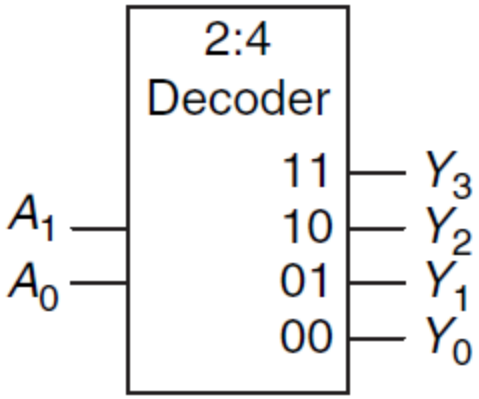



**Figure 2.58** 4:1 multiplexer implementations: (a) two-level logic, (b) tristates, (c) hierarchical

# Combinational Building Blocks

**Decoders**

- A decoder has **N** inputs and $\mathbf{2^N}$ outputs. It asserts exactly one of its outputs depending on the input combination.
- **Figure 2.63** shows a 2:4 decoder. When A1:0 = 00, Y0 is 1. When A1:0 = 01, Y1 is 1. And so forth.
- The outputs are called one-hot, because exactly one is “hot”(HIGH) at a given time.




| $A_1$ | $A_0$ | $Y_3$ | $Y_2$ | $Y_1$ | $Y_0$ |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 0 | 1 | 0 | 0 |
| 1 | 1 | 1 | 0 | 0 | 0 |

**Figure 2.63 2:4 decoder**

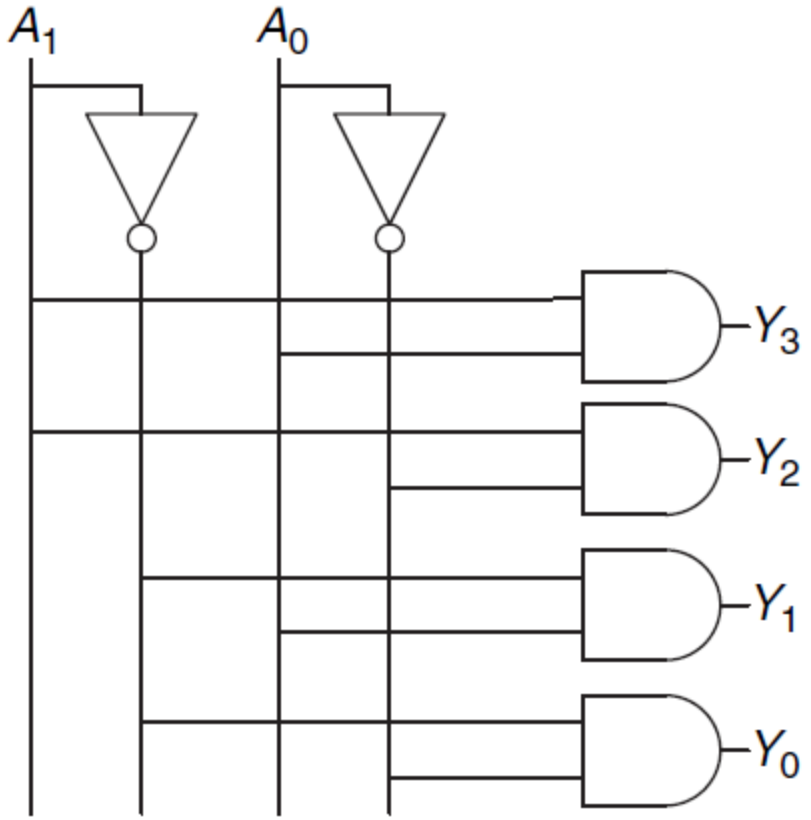



**Figure 2.64 2:4 decoder implementation**